

Labwork 7: Reduction

Philippe Olivier Schriqui-Duque
ID: 2640042

Introduction

In this labwork I implemented a grayscale stretch on the GPU. It is done in 3 steps: convert the image to gray (map), find the minimum and maximum intensity (reduce), and recalculate every pixel from $[min, max]$ to $[0, 255]$ (map). My 4K photo (3840×2880) already used the full range, so I reduced its contrast first to see the effect: its gray values are between 57 and 181.

Step 1: Grayscale (map)

This is my grayscale kernel from Labwork 4, but the output has only one channel of shape (h, w) , because only one gray value per pixel is needed for the next steps.

```
dst[y, x] = (int(src[y, x, 0]) +
             int(src[y, x, 1]) +
             int(src[y, x, 2])) // 3
```

Step 2: Min and max (reduce)

The gray image is seen as a 1D array of $n = h \times w$ pixels. Each block of 256 threads copies its part of the image into shared memory, then reduces it by pairs until the first cell contains the minimum (and the maximum) of the block. After each step, `cuda.syncthreads()` waits for the whole block. Thread 0 writes the result of the block. Threads outside the image put a neutral value (255 for the min, 0 for the max).

The kernel gives one result per block, so I copy these results back and finish with `min()` and `max()` on the CPU. It is only 21600 values, so it is very fast.

Naive version

This is the version from the slides, where the stride s goes up (1, 2, 4...):

```
s = 1
while s < cuda.blockDim.x:
    if localtid % (s * 2) == 0:
        cache_min[localtid] = min(cache_min[localtid],
                                   cache_min[localtid + s])
        # same with max() for cache_max
    cuda.syncthreads()
    s = s * 2
```

The condition `localtid % (s * 2) == 0` causes branch divergence: inside a warp, some threads work and the others wait.

Optimized version

I applied the optimizations from the slides:

- The stride goes down ($s = 128, 64, \dots 1$) and only threads with `localtid < s` work. These threads are next to each other, so whole warps are either working or idle (no branch divergence), and thread i reads cells i and $i + s$ (no bank conflicts).
- Each thread loads 2 pixels and already keeps the min and max of both before filling the cache. A block of 256 threads then handles 512 pixels, so I launch half as many blocks and no thread is idle in the first step.

```
s = cuda.blockDim.x // 2
while s > 0:
    if localtid < s:
        cache_min[localtid] = min(cache_min[localtid],
                                   cache_min[localtid + s])
        # same with max() for cache_max
    cuda.syncthreads()
    s = s // 2
```

Step 3: Stretch (map)

Each pixel is recalculated with the formula from the slides:

$$g' = \frac{g - \min}{\max - \min} \times 255$$

```
dst[y, x] = (src[y, x] - mn) / (mx - mn) * 255
```

The gray image stays on the GPU during the 3 steps, there is no copy in the middle.

Results

The reduction finds a minimum of 57 and a maximum of 181, which is the same as `numpy`. After the stretch, the gray values go from 0 to 255 and the image has a much stronger contrast (see `gray.jpg` and `stretched.jpg`).

To measure the effect of the optimizations, I ran both reductions 20 times on the same gray image already on the GPU:

```
reduction naive:      1.557 ms
reduction optimized: 0.976 ms
speedup: 1.60x
```

The optimized reduction is 1.6 times faster, thanks to the removal of the branch divergence and the bank conflicts, and to the 2 pixels loaded by each thread.

Block size vs time

The block size is used by the two map kernels (the reduction always uses 1D blocks of 256 threads). I ran the whole pipeline 20 times for each 2D block size:

8x8	10.37 +/- 0.77 ms
16x8	10.62 +/- 1.40 ms
16x16	9.73 +/- 1.05 ms
32x8	11.82 +/- 6.08 ms
16x32	12.03 +/- 1.18 ms
32x16	12.83 +/- 1.75 ms
32x32	11.96 +/- 1.38 ms

16x16 is the fastest, but the differences stay small. Like in the previous labworks, most of the time is spent copying the image to the GPU and back, not in the kernels. The 32x8 result has a big standard deviation (6.08 ms), so it is affected by a few slow runs.

Conclusion

The grayscale stretch combines the two patterns of the course: two maps and one reduction. The reduction is harder to write because the threads of a block have to work together with shared memory and synchronization. The optimizations from the slides made it 1.6 times faster, while the 2D block size has little effect on the whole program.